

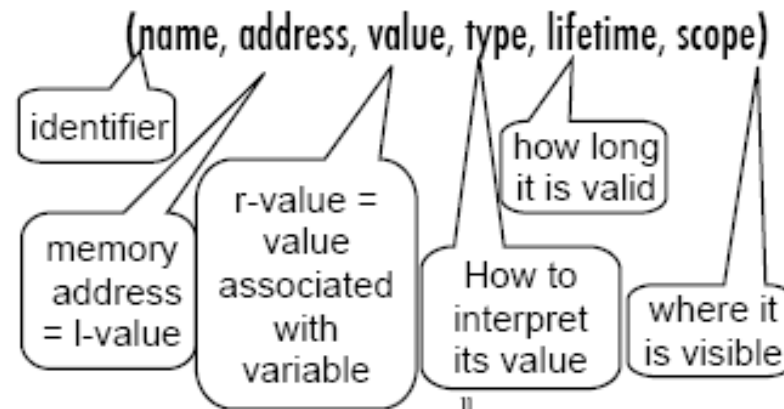
Chapter 5

Names, Bindings, and Scopes

Variables

Variables

- A **variable** is an abstraction of a **memory cell**
- Variables can be characterized by a set of attributes:
 - Name
 - Address
 - Value
 - Type
 - Lifetime
 - Scope



Names

- Design issues for names:
 - Are names case sensitive?
 - Are special words reserved words or keywords?

Type Binding

- Type determines the range of values of variables and the set of operations that are defined for values of that type
- Two aspects
 - How is a type specified?
 - When does type binding take place?

Static and Dynamic Type Binding

- A type binding is *static* if it first occurs **before run time and remains unchanged** throughout program execution.
- A type binding is *dynamic* if it first occurs **during execution or can change** during execution of the program

Static type binding:

Explicit/Implicit Declaration

- An *explicit declaration* is a program statement used for declaring the types of variables
- An *implicit declaration* is a default mechanism for specifying types of variables through default conventions, rather than declaration statements

With Fortran, If the identifier begins with one of the letters I, J, K, L, M, or N, or their lowercase versions, it is implicitly declared to be Integer type; otherwise, it is implicitly declared to be Real type.

Static type binding:

Explicit/Implicit Declaration

- Some languages use **type inferencing** to determine types of variables (context)

- Visual BASIC 9.0+, ML, Haskell, F#, and Go
- the context of the appearance of a variable determines its type

// This statement in c#:

```
var foo = "bar";
```

// Is equivalent to this statement:

```
string foo = "bar"
```

in ML, the result of the function is real based on constant type.

```
fun circumf(r) = 3.1415926 * r * r
```

Dynamic Type Binding

- Dynamic Type Binding
 - JavaScript, Python, Ruby, PHP
- Specified through an assignment statement
 - e.g., JavaScript

```
list = [2, 4.33, 6, 8];  
list = 17.3;
```


Storage binding and life time

Storage Bindings & Lifetime

- Storage Bindings
 - Allocation - getting a cell from some pool of available cells
 - Deallocation - putting a cell back into the pool
- The **lifetime** of a variable is the time during which it is bound to a particular memory cell

Categories of Variables by Lifetime

- Static
- Stack Dynamic
- Explicit Heap Dynamic
- Implicit Heap Dynamic

Static variables

- **Static**--bound to memory cells before execution begins and remains bound to the same memory cell throughout execution.
 - Ex: C static variables

```
/*  Function reads an integer, and keeps cumulative sum of
    integer read and the previous value of a static variable sum.
*/
void sumit(void)
{
    static int sum = 0;          /* sum is initialized to zero  */
                                /* at compile time. */
    int num;

    printf("Enter a number: ");
    scanf("%d",&num);
    sum += num;
    printf("The current total is %d\n",sum);
}
```

Stack-dynamic

- **Stack-dynamic**--Storage bindings are created for variables when their declaration statements are elaborated.
 - In Java, C++, and C#, variables defined in subprograms are by default stack-dynamic

```
/*  Function reads an integer, and keeps cumulative sum of
    integer read and the previous value of a static variable sum.
*/
void sumit(void)
{   static int sum = 0;           /* sum is initialized to zero  */
                                   /* at compile time. */
    int num;

    printf("Enter a number: ");
    scanf("%d",&num);
    sum += num;
    printf("The current total is %d\n",sum);
}
```

Explicit heap-dynamic variables

- *Explicit heap-dynamic* -- Allocated and deallocated by **explicit directives**, specified by the programmer, which take effect during execution

e.g. dynamic objects in C++ (via **new** and **delete**), all objects in Java

```
Employee e = new Employee();
```

Implicit heap-dynamic variables

- *Implicit heap-dynamic*--allocation and deallocation caused by assignment statements
 - all variables in APL;
 - all strings and arrays in Perl and JavaScript
 - `list = [10.2, 3.5];`
 - `list = 47;`

Summary

Lifetime	Binding starts	Binding ends
Static	program start	program end
Stack-dynamic	evaluation of declaration	return to callee
Explicit heap-dynamic	explicit command (allocation)	explicit command (deallocation)
Implicit heap-dynamic	when assigned a value	when assigned another value

statics in Java

Block-structured languages

Ada, C, C++

Perl, JavaScript

Scope

Scope

- A scope is an enclosing context.

```
namespace
{
    class C
    {
        void f (bool b)
        {
            if (b)
            {
                ...
            }
        }
    };
}
```

// namespace scope, merely groups identifiers

// class scope, defines/declares member variables and functions

// function scope, contains executable statements

// unnamed scope for conditionally executed statements

Scope of a variable

- Is the **range of statements** over which it is visible
- A **variable is visible** in a statement if it can be referenced in it.
- **Scope rules** determine how **occurrences** of a name are associated with a variable.
- One of the basic reasons for scoping is to keep variables in different parts of the program distinct from one another.

Static Scope

- The scope of a variable is determined based on physical structure of program
- With static scoping, a variable always refers to its **nearest enclosing unit**
- **Search process**: search declarations, first locally, then in increasingly larger enclosing scopes, until one is found for the given name

Static Scope Example_(Ada)

```
procedure Big is
  X : Integer;
  ...
  procedure Sub1 is
    begin -- Sub1
    ...X... ←
    end; -- Sub1

  procedure Sub2 is
    X : Integer;
    begin -- Sub2
    ...X... ←
    end; -- Sub2

begin -- Big
  ...
end; -- Big
```

Dynamic Scope

- The scope of a variable is determined based on **calling sequences** of program units, not their textual layout
- **Search process**: search declarations, first locally, then the dynamic parent, or calling procedure, until one is found for the given name

Dynamic Scope Example

```
procedure Big is
  X : Integer;
  ...
  procedure Sub1 is
    begin -- Sub1
    ...X...
    end; -- Sub1

  procedure Sub2 is
    X : Integer;
    begin -- Sub2
    ...X...
    end; -- Sub2

begin -- Big
  ...
end; -- Big
```

X reference in Sub 1:

1. Big calls Sub2, which calls Sub1
2. Big calls Sub 1 directly

Scope Example

```
function big() {  
  function sub1(){  
    var x = 7;  
    sub2();  
  }  
  function sub2() {  
    var y = x;  
  }  
  var x = 3;  
  sub1();  
}
```

big calls sub1
sub1 calls sub2
sub2 uses x

– Static scoping

- Reference to x in sub2 is to big's x

– Dynamic scoping

- Reference to x in sub2 is to sub1's x

Scope ≠ Lifetime

- Scope and lifetime are sometimes closely related, but are **different** concepts
- A variable may be **alive** although **not in scope**.
- Examples:
 - **static variables**: always alive but not always visible.
 - **shadowed variables**:
 - a variable name shadowed by a formal parameter of the same name (in Java use `this.name` to avoid shadowing of name)

Referencing Environments

- The *referencing environment of a statement* is the collection of all names that are visible in the statement
- In a static-scoped language, it is the local variables plus all of the visible variables in all of the enclosing scopes
- In a dynamic-scoped language, the referencing environment is the local variables plus all visible variables in all active subprograms

Dynamic-scoped

```
void sub1() {  
    int a, b;  
    ... ←———— 1  
} /* end of sub1 */  
void sub2() {  
    int b, c;  
    .... ←———— 2  
    sub1();  
} /* end of sub2 */  
void main() {  
    int c, d;  
    ... ←———— 3  
    sub2();  
} /* end of main */
```

The referencing environments of the indicated program points are as follows:

<i>Point</i>	<i>Referencing Environment</i>
1	a and b of sub1, c of sub2, d of main, (c of main and b of sub2 are hidden)
2	b and c of sub2, d of main, (c of main is hidden)
3	c and d of main